

The Stress Terminal UI: s-tui



pypi package
 1.4.0
 downloads/month
 5k



Stress-Terminal UI, s-tui, monitors CPU temperature, frequency, power and utilization in a graphical way from the terminal.

Screenshot



Table of Contents



- [The Stress Terminal UI: s-tui](#)
 - [Screenshot](#)
 - [What it does](#)
 - [Usage](#)
 - [Simple installation](#)
 - [pip \(x86 ARM\)](#)
 - [More installation methods](#)
 - [Ubuntu \(18.10 and newer\)](#)
 - [Ubuntu \(18.04, 16.04\)](#)
 - [Debian \(10 and newer\)](#)
 - [Arch Linux, Manjaro](#)
 - [OpenSUSE](#)
 - [Fedora](#)
 - [Options](#)
 - [Throttle Indicators](#)
 - [Dependencies](#)
 - [Configuration](#)
 - [Saving a configuration](#)
 - [Adding threshold scripts](#)
 - [Run from source code](#)
 - [Compatibility](#)
 - [FAQ](#)
 - [Development](#)
 - [Contributing](#)
 - [Tip](#)

What it does



- Monitoring your CPU temperature/utilization/frequency/power
- Shows performance dips caused by thermal throttling
- Requires no X-server
- Built-in CPU stress test (zero dependencies, works out of the box)
- Optional integration with external stress tools (stress/stress-ng)

Usage



```
s-tui
```

Simple installation



pip (x86 + ARM)



The most up to date version of s-tui is available with pip.

Install with:

```
pip install s-tui --user
```

(This usually creates an executable in `~/.local/bin/` dir. Make sure it is in your PATH)

To install as root

```
sudo pip install s-tui
```

You might need to install `python-dev` first

Installation in virtualenv with [pipsi](#):

```
pipsi install s-tui
```

More installation methods



Ubuntu (18.10 and newer)



```
sudo apt install s-tui
```

Ubuntu (18.04, 16.04)



A PPA is available but is not up to date

```
sudo add-apt-repository ppa:amanusk/python-s-tui
sudo apt-get update
sudo apt-get install python3-s-tui
```

Debian (10 and newer)



```
sudo apt install s-tui
```

Arch Linux, Manjaro



s-tui is in the Arch repository:

```
sudo pacman -S s-tui
```

s-tui-git follows the master branch, maintained by [@MauroMombelli](#)

Install it with: `yay -S s-tui-git`

OpenSUSE



```
sudo zypper install s-tui
```

Fedora



s-tui is in the Fedora [repository](#):

```
sudo dnf install s-tui
```

Options



TUI interface:

The side bar houses the controls for the displayed graphs.

At the bottom, all sensors reading are presented in text form.

- * Use the arrow keys or 'hjkl' to navigate the side bar
- * Toggle between stressed and regular operation using the radio buttons in 'Modes'.
- * If you wish to alternate stress defaults, you can do it in <Stress options>
- * Select graphs to display in the <Graphs> menu
- * Select summaries to display in the <Summaries> menu
- * Use the <Reset> button to reset graphs and statistics
- * If your system supports it, you can use the UTF-8 button to get a smoother graph
- * Save your current configuration with the <Save Settings> button
- * Press 'q' or the <Quit> button to quit

* Run ``s-tui --help`` to get this message and additional cli options

optional arguments:

- | | |
|--|---|
| <code>-h, --help</code> | show this help message and exit |
| <code>-d, --debug</code> | Output debug log to <code>_s-tui.log</code> |
| <code>--debug-file DEBUG_FILE</code> | Use a custom debug file. Default: <code>_s-tui.log</code> |
| <code>-dr, --debug_run</code> | Run for 5 seconds and quit |
| <code>-c, --csv</code> | Save stats to csv file |
| <code>--csv-file CSV_FILE</code> | Use a custom CSV file. Default: <code>s-tui_log_<TIME>.csv</code> |
| <code>-t, --terminal</code> | Display a single line of stats without tui |
| <code>-j, --json</code> | Display a single line of stats in JSON format |
| <code>-nm, --no-mouse</code> | Disable Mouse for TTY systems |
| <code>-v, --version</code> | Display version |
| <code>-tt T_THRESH, --t_thresh T_THRESH</code> | High Temperature threshold. Default: 80 |

Throttle Indicators



When CPU throttling is detected, s-tui changes the frequency graph and summary text color and appends a reason label. Labels may be combined with / (e.g. T/W).

With root + msr module (detailed per-core reasons via IA32_THERM_STATUS):

Label	Meaning
T	Thermal — core temperature exceeded TjMax
H	PROCHOT — external thermal signal (VRM, GPU, battery)
C	Critical — near emergency shutdown temperature
W	Power limit — PL1/PL2 watt budget exceeded
A	Current limit — electrical current (amps) limit hit
X	Cross-domain — throttled by another domain (e.g. GPU)

Without root (sysfs fallback, thermal only):

Label	Meaning
Tc	Core thermal throttle
Tp	Package thermal throttle (affects all cores)

When a temperature sensor exceeds the high value defined by the sensor, or a user configured threshold (default 80°C), the sensor summary text summary and associated graph will change color to indicate this.

Dependencies



s-tui includes a built-in CPU stress test that works out of the box with no extra dependencies. For better stress performance, install numpy:

```
pip install s-tui[stress]
```

or install numpy directly:

```
pip install numpy
```

When numpy is available, the built-in stresser uses mixed floating-point operations (multiply, sqrt, sin) for maximum thermal output. Without numpy, it falls back to hashlib SHA-256 hashing which still provides good CPU load.

For additional stress options (memory, I/O, sync workers), you can optionally install the external stress or stress-ng tool:

```
sudo apt-get install stress
```

Configuration



s-tui is a self-contained application that can run out-of-the-box and doesn't need config files to drive its core features. However, additional features like running scripts when a certain threshold has been exceeded (e.g. CPU temperature) does necessitate creating a config directory. This directory will be made in `~/.config/s-tui` by default.

Saving a configuration



Selecting <Save Settings> will save the current configuration to `~/.config/s-tui/s-tui.conf`. If you would like to restore defaults, simply remove the file.

Adding threshold scripts



s-tui gives you the ability to run arbitrary shell scripts when a certain threshold is surpassed, like your CPU temperature. You can define this custom behaviour by adding a shell file to the directory `~/.config/s-tui/hooks.d` with one of the following names, depending on what threshold you're interested in reacting to:

- `tempsource.sh`: triggered when the CPU temperature threshold is exceeded

If s-tui finds a script in the hooks directory with the name of a source it supports, it will run that script every 30 seconds as long as the current value of the source remains above the threshold.

Note that at the moment only CPU temperature threshold hooks are supported.

Run from source code



Start by cloning the repository

```
git clone https://github.com/amanusk/s-tui.git
cd s-tui
```

Install required dependencies as [root] or as (local user)

```
[sudo] pip install urwid (--user)
[sudo] pip install psutil (--user)
```

Install stress (optional)

```
sudo apt-get install stress
```

Run the .py file

```
python -m s_tui.s_tui
```

Compatibility



s-tui uses [psutil](#) to probe hardware information. If your hardware is not supported, you might not see all the information.

s-tui uses [urwid](#) as a graphical engine. urwid only works with UNIX-like systems

- Power read is supported on Intel Core CPUs of the second generation and newer (Sandy Bridge) and on AMD Family 17h CPUs through the [amd_energy](#) driver.
- s-tui tested to run on Raspberry-Pi 4,3,2,1

FAQ



Q: How is this different from htop?

A: s-tui is not a processes monitor like htop. The purpose is to monitor your CPU statistics and have an option to test the system under heavy load. (Think AIDA64 stress test, not task manager).

Q: I am using the TTY with no X server and s-tui crashes on start

A: By default, s-tui handles mouse inputs. This causes some systems to crash. Try running `s-tui --no-mouse`

Q: I am not seeing all the stats in the sidebar.

A: The sidebar is scrollable, you can scroll down with `DOWN` or `j` or scroll to the bottom with `PG-DN` or `G`. You might consider also decreasing the size of the font that you use in your terminal.:)

Development



Install dev dependencies:

```
pip install -e ".[test]"
pip install ruff==0.15.4 pyright==1.1.408
```

Set up pre-commit hooks (runs ruff + pyright automatically before each push):

```
pip install pre-commit
pre-commit install --hook-type pre-push
```

Run checks manually:

```
ruff check .          # lint
ruff format --check . # format check
pyright s_tui/        # type check
pytest               # tests
```

Auto-fix lint and formatting:

```
ruff check --fix .
ruff format .
```

Contributing



New issues and Pull Requests are welcome :)

If you notice a bug, please report it as a new issue, using the provided template.

To open a Pull Request, please see [CONTRIBUTING](#) for more information.

Tip



If you like this work, please star it on GitHub.

BTC: 1PPhYgecwvAN7utN2EotgTfy2mmLqzF8m3

ETH: 0xc169699A825066f2F07E0b29C4082094b32A3F3e